

Admin and Analytics APIs

Module 9 • Day 2 (Hands-On + Demonstrations)

Cursor Training Program • ~75 min

Module Overview

Aspect		Details
Duration	~75 minutes	
Format	Hands-on exercise + demonstrations	
Prerequisites	Admin API key (not User key), Python 3.8+, Modules 7–8 completed	
Module Goal	Master team management, usage analytics, cost governance, and safe admin operations	

Learning Objectives

By the end of this module, participants will be able to:

- List and manage team members programmatically
- Retrieve daily usage data for cost tracking and reporting
- Set user spend limits for budget governance
- Analyze model usage for cost optimization insights
- Track daily active users for leadership reporting
- Build responsible leaderboards without privacy violations
- Analyze conversation intent and complexity (demonstration)
- Safely remove team members with proper patterns (demonstration)

Agenda

Lesson	Topic	Time	Type
9.1	Listing Team Members	8 min	Exercise
9.2	Daily Usage Data	10 min	Exercise
9.3	Setting User Spend Limits	8 min	Exercise
9.4	Model Usage Analytics	8 min	Exercise
9.5	Daily Active Users	6 min	Exercise
9.6	Leaderboards	6 min	Exercise
9.7	Conversation Insights	6 min	Demo
9.8	Destructive Admin Operations	6 min	Demo

Lesson 9.1

Listing Team Members

Concept · 5 min · Exercise · 8 min

User vs. Admin API Key

Aspect	User API Key	Admin API Key
Scope	Your user only	Entire organization
Can list members	✗ No	✓ Yes
Can view others' usage	✗ No	✓ Yes
Can modify policies	✗ No	✓ Yes
Format	cursor_xxx...	cursor_admin_xxx...

Key endpoint: `GET /v1/admin/members`

Exercise 9.1 – Setup & List

```
export CURSOR_ADMIN_API_KEY="cursor_admin_XXXXXXXXXX"

# Verify admin access
curl -s -u "$CURSOR_ADMIN_API_KEY:" \
  https://api.cursor.com/v1/admin/organization | jq '.'

# List all team members
curl -s -u "$CURSOR_ADMIN_API_KEY:" \
  "https://api.cursor.com/v1/admin/members" | jq '.'
```

Exercise 9.1 – Pagination & Export

Pagination:

```
curl -s -u "$CURSOR_ADMIN_API_KEY:" \
  "https://api.cursor.com/v1/admin/members?limit=10&offset=0"
```

Python: loop with offset until empty → export to `team_roster.csv` (email, role, status, joined, lastActiveAt)

Helper: `get_user_id_by_email(email)` for downstream admin calls

Success Criteria: Authenticated • listed members • handled pagination • exported CSV

Lesson 9.2

Daily Usage Data

Concept · 5 min · Exercise · 10 min

Key Endpoint

```
GET /v1/admin/analytics/usage/daily
```

Returns:

- Cost per day • Input/output token counts
- Active users per day • Breakdown by user and model (optional)

“ *"Finance asks: 'What did we spend yesterday?' Engineering leads ask: 'Who's using what?'"* ”

Exercise 9.2 – Weekly Usage

```
END=$(date +%Y-%m-%d)
START=$(date -d "7 days ago" +%Y-%m-%d)

curl -s -u "$CURSOR_ADMIN_API_KEY:" \
  "https://api.cursor.com/v1/admin/analytics/usage/daily?startDate=$START&endDate=$END" \
  | jq '.daily[] | {date: .date, cost: .cost, tokens: .totalTokens, users: .activeUsers}'
```

Exercise 9.2 – Cost Report

Python `generate_cost_report()` for last 30 days:

- Total cost · total tokens · average daily cost/users
- Week-over-week change · top 5 costliest days
- Daily breakdown table (last 14 days)
- Budget alerts at \$300 / \$500 thresholds

Success Criteria: Retrieved date range · calculated trends · generated readable report

Lesson 9.3

Setting User Spend Limits

Concept · 5 min · Exercise · 8 min

Key Endpoint

PATCH /v1/admin/policies/users/{userId}/limits

Action	Behavior
alert	Send notification but allow usage
block	Prevent any further requests for the month

Exercise 9.3 – Set Limits

```
USER_ID=$(curl -s -u "$CORSOR_ADMIN_API_KEY:" \
  "https://api.cursor.com/v1/admin/members?email=developer@company.com" \
  | jq -r '.members[0].id')

curl -X PATCH ".../policies/users/$USER_ID/limits" \
  -u "$CORSOR_ADMIN_API_KEY:" \
  -d '{"monthlyLimit": 50.00, "exceedanceAction": "block"}'
```

Check current limit: `GET .../policies/users/{userId}/limits`

Exercise 9.3 – Bulk Limits

CSV bulk set: `email, monthly_limit, action`

```
intern@company.com,20,block  
contractor@company.com,50,alert  
lead@company.com,200,alert
```

Find heavy users: query `/analytics/usage/users` for current month → filter cost > threshold

Success Criteria: Retrieved user ID · set limit · verified · bulk setting implemented

Lesson 9.4

Model Usage Analytics

Concept · 5 min · Exercise · 8 min

Key Endpoint

```
GET /v1/admin/analytics/usage/models
```

“ *"Which models are actually being used? Is Opus worth the cost? Should you train people on cheaper alternatives?"* ”

Exercise 9.4 – Model Breakdown

```
curl -s -u "$CURSOR_ADMIN_API_KEY:" \
  ".../analytics/usage/models?startDate=$START&endDate=$END" \
  | jq '.models[] | {model: .modelId, cost: .cost, requests: .requestCount}'

# Find Opus overuse per user
curl -s -u "$CURSOR_ADMIN_API_KEY:" \
  ".../analytics/usage/users?startDate=$START&endDate=$END" \
  | jq '.users[] | select(.modelBreakdown."claude-4.7-opus" != null)'
```

Exercise 9.4 – Optimization Report

`generate_optimization_report()`:

- Model cost breakdown (% of total)
- Users on Claude Opus → suggest Sonnet for non-critical tasks
- High Sonnet usage → suggest GPT-5.3 Codex (40% savings)
- Estimated monthly savings if guidelines applied

Success Criteria: Retrieved model breakdown • identified expensive users • generated recommendations

Lesson 9.5

Daily Active Users (DAU)

Concept · 4 min · Exercise · 6 min

Why DAU Matters

- Track adoption after rollout
- Identify unused licenses for reallocation
- Measure impact of training sessions
- Justify renewal and expansion

Source: `activeUsers` field from `/admin/analytics/usage/daily`

Exercise 9.5 – DAU Report

```
curl -s -u "$CURSOR_ADMIN_API_KEY:" \
  ".../analytics/usage/daily?startDate=$START&endDate=$END" \
  | jq '{avg_weekly: ([.daily[-7:] | .[].activeUsers] | add / length),
      peak: ([.daily[] | .activeUsers] | max)}'
```

Python leadership report:

- Average/median/peak DAU • adoption rate (% of team)
- WoW growth rate • weekly averages
- Health assessment: >80% excellent • >50% good • <30% investigate

Success Criteria: Calculated DAU • adoption metrics • leadership-ready report

Lesson 9.6

Leaderboards

Concept · 5 min · Exercise · 6 min

Responsible Leaderboard Principles

Principle	Implementation
Anonymize	Roles or anonymized names, not full emails
Focus on positive metrics	Show savings, not spending
Opt-in only	Allow users to choose public visibility
Include context	Show team size, role differences

Exercise 9.6 – Three Leaderboards

1. **Engagement leaderboard** – rank by request count (anonymized emails)
2. **Efficiency leaderboard** – tokens per dollar spent
3. **Savings leaderboard** – users who saved by choosing efficient models over Opus

```
def anonymize_email(email):  
    local = email.split('@')[0]  
    return local[:2] + "... " + local[-2:]
```

Success Criteria: Anonymized · efficiency-focused · savings-focused leaderboards

Lesson 9.7

Conversation Insights

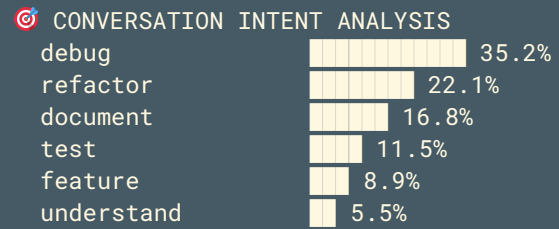
Concept · 6 min · Demonstration

What Conversation Insights Reveal

- Simple questions vs. complex refactors?
- Most common task categories
- Where users get stuck
- Which models perform best for which task types

Endpoint: `GET /v1/admin/analytics/conversations` (may require Enterprise plan)

Demo: Intent Analysis



Demo: Complexity & Categories

Complexity distribution:

- simple 45% · moderate 33% · complex 15% · architectural 7%

Category analysis:

- backend 40% · frontend 29% · database 15% · devops 10% · security 6%

Stuck patterns: conversations >5 min with `success: false` → suggest training/docs

Success Criteria: Understood capabilities · intent/complexity/category tracking · stuck patterns

Lesson 9.8

Destructive Admin Operations

Concept · 6 min · Demonstration

Safe Removal Playbook

1. **Audit first** — active agents, runs, API keys
2. **Soft delete** — deactivate (no new agents; existing continue)
3. **Transfer ownership** — critical agents, webhooks
4. **Log everything** — compliance audit trail
5. **Confirm before hard delete** — GDPR/security only

Demo: SafeRemovalDemo Workflow

```
Step 1: find_user(email) → user_id  
Step 2: audit_resources() → agents, runs  
Step 3: deactivate() → status: inactive  
Step 4: transfer_resources(new_owner_email)  
Step 5: hard_delete() → permanent (rare)  
→ generate_audit_report()
```

Bulk deactivation: find users inactive 90+ days → review → notify → deactivate

Success Criteria: 5-step pattern · audit-first · soft vs hard delete · resource transfer

Module Summary

Lesson	Topic	Type
9.1	Listing Team Members	Exercise
9.2	Daily Usage Data	Exercise
9.3	Setting User Spend Limits	Exercise
9.4	Model Usage Analytics	Exercise
9.5	Daily Active Users	Exercise
9.6	Leaderboards	Exercise
9.7	Conversation Insights	Demo
9.8	Destructive Operations	Demo

Quick Reference Card

ENDPOINTS:

GET	/admin/members	List members
GET	/admin/analytics/usage/daily	Daily usage
GET	/admin/analytics/usage/models	Model usage
GET	/admin/analytics/usage/users	Per-user usage
PATCH	/admin/policies/users/{id}/limits	Set spend limits
GET	/admin/members/{id}/resources	User resources

LEADERBOARDS: Anonymize · positive metrics · opt-in · role context

REMOVAL: Audit → Deactivate → Transfer → Log → Confirm delete

Up Next: Module 10

AI Code Tracking and Reporting

“ Now that you can manage your team and analyze usage, **Module 10** covers attributing AI contributions per commit, exporting metrics to BI tools, and building complete dashboards. ”

End of Module 9